Day 6 Task Workflow — Sunday, Mar 2, 2026

> **Sprint:** 1 (Foundation) | **Phase:** Backend Services & WebSocket Infrastructure
> **Owner:** Lead Architect | **Status:** ⬤ Ready to Start
> **Goal:** Flow service, Alert service, WebSocket server, frontend hooks, Week 1 completion

---

Day 6 Objective

Complete the remaining Week 1 foundation tasks and begin Week 2 preparation so that by end of day:

- Flow service implemented with CRUD endpoints
- Alert service implemented with lifecycle management
- WebSocket server operational with Redis pub/sub integration
- Frontend hooks: useWebSocket, useFlows, useAlerts
- War Room components connected to live API data
- Week 1 demo ready: beautiful dark dashboard with live data

Scope Adherence Checklist

> **STRICT RULE:** Every task below MUST adhere to the master documentation specifications. No features outside the defined scope.

Requirement	Source Document	Section
Flow Service CRUD	MASTER_DOC_PART2	§5.1 (`/flows/*` endpoints)
Alert Service lifecycle	MASTER_DOC_PART2	§5.1 (`/alerts/*` endpoints)
WebSocket + Redis Pub/Sub	MASTER_DOC_PART2	§6.1 (Real-Time Communication)
RBAC enforcement	MASTER_DOC_PART2	§7.2 (Role Permissions Matrix)
War Room components	MASTER_DOC_PART3	§2.3 (Components Specification)
useWebSocket hook	MASTER_DOC_PART5	§2.1 (hooks/useWebSocket.ts)
useFlows hook	MASTER_DOC_PART5	§2.1 (hooks/useFlows.ts)
useAlerts hook	MASTER_DOC_PART5	§2.1 (hooks/useAlerts.ts)
API response time <200ms	MASTER_DOC_PART1	§10.1 (Technical KPIs)

Task Breakdown

TASK 1 — Flow Service Implementation ⬤

Time Est: 45 min | **Priority:** ⬤ Critical

Implement the flow data service with CRUD operations and aggregation queries.

1.1 Create Flow Service (`backend/app/services/flow_service.py`)

Function	Purpose	Endpoint
`get_flows()`	List flows with pagination, filtering	`GET /flows/`
`get_flow_by_id()`	Single flow detail	`GET /flows/{id}`
`get_flow_stats()`	Aggregated statistics	`GET /flows/stats`
`get_top_talkers()`	Top IPs by volume	`GET /flows/top-talkers`
`get_protocol_distribution()`	Protocol breakdown	`GET /flows/protocols`
`search_flows()`	Advanced search	`POST /flows/search`

Implementation Requirements (per MASTER_DOC_PART2 §5.1):

- Pagination: `page` and `limit` query params (default: page=1, limit=50)
- Filtering: `time_range`, `src_ip`, `dst_ip`, `protocol`, `is_anomaly`, `min_score`
- Sorting: `timestamp DESC` by default
- RBAC: All authenticated users can view flows

Database Queries:

```
# Flow statistics aggregation
SELECT
    COUNT(*) as total_flows,
    COUNT(*) FILTER (WHERE is_anomaly = true) as anomaly_count,
    AVG(anomaly_score) as avg_score,
    SUM(total_bytes) as total_bytes,
    SUM(total_packets) as total_packets
FROM network_flows
WHERE timestamp > NOW() - INTERVAL '{interval}'

# Top talkers
SELECT src_ip, COUNT(*) as flow_count, SUM(total_bytes) as bytes
FROM network_flows
WHERE timestamp > NOW() - INTERVAL '1 hour'
GROUP BY src_ip
ORDER BY flow_count DESC
LIMIT 10

# Protocol distribution
SELECT protocol, COUNT(*) as count
FROM network_flows
WHERE timestamp > NOW() - INTERVAL '1 hour'
GROUP BY protocol
ORDER BY count DESC
```

Verification:

Check	Command	Expected
Service imports	`python -c "from app.services.flow_service import *"`	No errors
Endpoints mounted	Check `/docs`	6 flow endpoints
Pagination works	`curl /api/v1/flows/?page=1&limit=10`	Paginated response
Stats aggregation	`curl /api/v1/flows/stats?interval=1h`	Aggregated metrics
Top talkers	`curl /api/v1/flows/top-talkers`	IP list with counts
Protocol distribution	`curl /api/v1/flows/protocols`	Protocol breakdown

TASK 2 — Alert Service Implementation 🔴

Time Est: 45 min | **Priority:** 🔴 Critical

Implement the alert management service with lifecycle operations.

2.1 Create Alert Service (`backend/app/services/alert_service.py`)

Function	Purpose	Endpoint
`get_alerts()`	List alerts with filters	`GET /alerts/`
`get_alert_by_id()`	Single alert detail	`GET /alerts/{id}`
`update_alert_status()`	Lifecycle transition	`PATCH /alerts/{id}/status`
`assign_alert()`	Assign to analyst	`PATCH /alerts/{id}/assign`
`get_alert_stats()`	Alert statistics	`GET /alerts/stats`
`create_alert()`	Auto-create from ML	Internal (ML Worker)

Alert Lifecycle (per MASTER_DOC_PART3 §7.1):

```
New Alert → OPEN → ACKNOWLEDGED → INVESTIGATING → RESOLVED
                                                  → FALSE_POSITIVE
```

Status Transition Rules:

From	To	Allowed Roles
open	acknowledged	admin, soc_manager, analyst

From	To	Allowed Roles
acknowledged	investigating	admin, soc_manager, analyst
investigating	resolved	admin, soc_manager
investigating	false_positive	admin, soc_manager, analyst
any	open	admin only

Severity Definitions (per MASTER_DOC_PART3 §7.3):

Level	Color	Confidence Threshold
critical	`#ef4444`	≥90%
high	`#f97316`	≥75%
medium	`#f59e0b`	≥50%
low	`#3b82f6`	≥30%
info	`#94a3b8`	<30%

Alert ID Format: `TM-ALERT-{sequence}` (e.g., `TM-ALERT-00001`)

Verification:

Check	Command	Expected
Service imports	`python -c "from app.services.alert_service import *"`	No errors
Endpoints mounted	Check `/docs`	5 alert endpoints
List alerts	`curl /api/v1/alerts/`	Alert list (empty initially)
Alert stats	`curl /api/v1/alerts/stats`	Statistics object
Status update	`PATCH /api/v1/alerts/{id}/status`	Updated alert
RBAC enforced	Viewer cannot resolve	403 Forbidden

TASK 3 — WebSocket Server Implementation 🔴

Time Est: 60 min | **Priority:** 🔴 Critical

Implement the WebSocket server with Redis pub/sub integration for real-time events.

3.1 Create WebSocket Module (`backend/app/api/v1/websocket.py`)

WebSocket Events (per MASTER_DOC_PART2 §5.2):

Channel	Event	Payload	Direction
`flows:live`	`new_flow`	Flow record with features	Server → Client
`alerts:live`	`new_alert`	Alert object	Server → Client
`alerts:live`	`alert_updated`	Updated alert	Server → Client
`system:status`	`capture_status`	Engine status	Server → Client
`system:status`	`system_metrics`	CPU/mem/throughput	Server → Client
`ml:live`	`anomaly_detected`	Anomaly with score	Server → Client
`llm:stream`	`token`	Streaming LLM response	Server → Client

Architecture (per MASTER_DOC_PART2 §6.1):

```
Capture Engine ──► Redis Pub/Sub ──► FastAPI ──► WebSocket ──► Browser
                   (flows:live)      (subscriber)  (broadcast)  (N clients)

ML Worker ────► Redis Pub/Sub ──► FastAPI ──► WebSocket ──► Browser
                (alerts:live)     (subscriber)  (broadcast)
```

Implementation Requirements:

- Endpoint: `/ws/` (WebSocket connection)
- Authentication: Token passed as query parameter `?token=<jwt>`
- Channel subscription: Client sends `{"action": "subscribe", "channels": ["flows:live", "alerts:live"]}`
- Heartbeat: Ping/pong every 30 seconds
- Reconnection: Client-side exponential backoff
- Broadcast: Redis subscriber listens to channels, broadcasts to all connected clients

Connection Flow:

```
# 1. Client connects to ws://localhost:8000/ws/?token=<jwt>
# 2. Server validates JWT token
# 3. Server adds client to connection pool
# 4. Client sends subscription request
# 5. Server subscribes to Redis channels
# 6. Redis events are broadcast to subscribed clients
# 7. On disconnect, cleanup subscriptions
```

Verification:

Check	Command	Expected
WebSocket endpoint	Check `/docs`	`/ws/` endpoint listed

Check	Command	Expected
Connection test	`wscat -c ws://localhost:8000/ws/?token=<jwt>`	Connected
Subscribe	Send `{"action":"subscribe","channels":["flows:live"]}`	Subscribed
Receive events	Publish to Redis `flows:live`	Client receives message
Auth required	Connect without token	Connection rejected
Heartbeat	Wait 30s	Ping/pong exchanged

TASK 4 — Frontend WebSocket Hook 🟡

Time Est: 30 min | **Priority:** 🟡 Medium

Implement the `useWebSocket` hook for real-time data streaming.

4.1 Update WebSocket Hook (`frontend/hooks/useWebSocket.ts`)

Hook Interface (per MASTER_DOC_PART5 §2.1):

```
interface UseWebSocketReturn {
  isConnected: boolean;
  systemStatus: SystemStatus | null;
  lastFlowEvent: FlowEvent | null;
  lastAlertEvent: AlertEvent | null;
  subscribe: (channels: string[]) => void;
  unsubscribe: (channels: string[]) => void;
  reconnect: () => void;
}
```

Features:

- Auto-connect on mount with JWT token
- Auto-reconnect with exponential backoff (1s, 2s, 4s, 8s, max 30s)
- Channel subscription management
- Event parsing and state updates
- Connection status indicator
- Heartbeat monitoring

WebSocket URL: `ws://localhost:8000/ws/?token=<jwt>`

Event Handling:

```
// Parse incoming messages
switch (data.event) {
  case 'new_flow':
    setLastFlowEvent(data.payload);
    break;
  case 'new_alert':
    setLastAlertEvent(data.payload);
    break;
  case 'capture_status':
  case 'system_metrics':
    setSystemStatus(prev => ({ ...prev, ...data.payload }));
    break;
}
```

Verification:

Check	Command	Expected
Hook exports	`grep "export.*useWebSocket" hooks/useWebSocket.ts`	Function exported
TypeScript compiles	`cd frontend && npx tsc --noEmit`	No errors
Connection logic	Read implementation	Auto-connect on mount
Reconnection logic	Read implementation	Exponential backoff
Channel management	Read implementation	subscribe/unsubscribe functions

TASK 5 — Frontend Flows Hook

Time Est: 20 min | **Priority:** Medium

Implement the `useFlows` hook for flow data queries.

5.1 Create Flows Hook (`frontend/hooks/useFlows.ts`)

Hook Interface:

```
interface UseFlowsReturn {
  flows: Flow[];
  stats: FlowStats | null;
  topTalkers: TopTalker[] | null;
  protocols: ProtocolDistribution[] | null;
  isLoading: boolean;
  error: string | null;
  refetch: () => void;
```

```
  searchFlows: (query: FlowSearchQuery) => Promise<Flow[]>;
}
```

API Endpoints Used:

Function	Endpoint	Method
`getFlows()`	`/api/v1/flows/?page=1&limit=50`	GET
`getStats()`	`/api/v1/flows/stats?interval=1h`	GET
`getTopTalkers()`	`/api/v1/flows/top-talkers`	GET
`getProtocols()`	`/api/v1/flows/protocols`	GET
`searchFlows()`	`/api/v1/flows/search`	POST

Features:

- Auto-refresh every 3 seconds (per MASTER_DOC_PART3 §2.3)
- Loading states
- Error handling
- Pagination support
- Filter parameters

Verification:

Check	Command	Expected
Hook exports	`grep "export.*useFlows" hooks/useFlows.ts`	Function exported
TypeScript compiles	`cd frontend && npx tsc --noEmit`	No errors
API integration	Read implementation	Uses api.ts client
Auto-refresh	Read implementation	setInterval with 3s

TASK 6 — Frontend Alerts Hook 🟡

Time Est: 20 min | **Priority:** 🟡 Medium

Implement the `useAlerts` hook for alert management.

6.1 Create Alerts Hook (`frontend/hooks/useAlerts.ts`)

Hook Interface:

```
interface UseAlertsReturn {
  alerts: Alert[];
  stats: AlertStats | null;
  isLoading: boolean;
  error: string | null;
```

```
  refetch: () => void;
  updateStatus: (alertId: string, status: AlertStatus) => Promise<void>;
  assignAlert: (alertId: string, userId: string) => Promise<void>;
}
```

API Endpoints Used:

Function	Endpoint	Method
`getAlerts()`	`/api/v1/alerts/?severity=&status=`	GET
`getStats()`	`/api/v1/alerts/stats`	GET
`updateStatus()`	`/api/v1/alerts/{id}/status`	PATCH
`assignAlert()`	`/api/v1/alerts/{id}/assign`	PATCH

Features:

- Auto-refresh every 5 seconds
- Severity filtering (critical, high, medium, low, info)
- Status filtering (open, acknowledged, investigating, resolved, false_positive)
- Optimistic UI updates
- Error handling with toast notifications

Verification:

Check	Command	Expected
Hook exports	`grep "export.*useAlerts" hooks/useAlerts.ts`	Function exported
TypeScript compiles	`cd frontend && npx tsc --noEmit`	No errors
Status update	Read implementation	PATCH request logic
Auto-refresh	Read implementation	setInterval with 5s

TASK 7 — War Room API Integration ◍

Time Est: 45 min | **Priority:** ◍ Medium

Connect War Room components to live API data using the new hooks.

7.1 Update War Room Page (`frontend/app/war-room/page.tsx`)

Components to Connect:

Component	Data Source	Hook	Refresh Rate
MetricCard (Pkts/s)	`/flows/stats`	useFlows	3s
MetricCard (Flows)	`/flows/stats`	useFlows	3s

Component	Data Source	Hook	Refresh Rate
MetricCard (Anomaly%)	`/flows/stats`	useFlows	3s
MetricCard (Threats)	`/alerts/stats`	useAlerts	5s
ProtocolChart	`/flows/protocols`	useFlows	10s
TrafficTimeline	`/flows/stats?interval=1m`	useFlows	5s
LiveAlertFeed	WebSocket `alerts:live`	useWebSocket	Real-time
TopTalkers	`/flows/top-talkers`	useFlows	10s
ThreatLevel	`/alerts/stats`	useAlerts	5s

Implementation:

```
'use client';

import { useFlows } from '@/hooks/useFlows';
import { useAlerts } from '@/hooks/useAlerts';
import { useWebSocket } from '@/hooks/useWebSocket';

export default function WarRoomPage() {
  const { stats: flowStats, protocols, topTalkers } = useFlows();
  const { stats: alertStats } = useAlerts();
  const { lastAlertEvent, systemStatus } = useWebSocket();

  // Calculate threat level from alert stats
  const threatLevel = calculateThreatLevel(alertStats);

  return (
    <div className="war-room-grid">
      {/* Components using live data */}
    </div>
  );
}
```

Verification:

Check	Command	Expected
Page compiles	`cd frontend && npx tsc --noEmit`	No errors
Hooks imported	Read page.tsx	useFlows, useAlerts, useWebSocket
Live data	Open `/war-room`	Data updates automatically
WebSocket events	Check browser console	Events received

TASK 8 — Week 1 Demo Verification

Time Est: 15 min | **Priority:** ⬛ Critical

Verify the Week 1 demo requirements are met.

8.1 Week 1 Demo Checklist (per MASTER_DOC_PART5 §3)

"End of Week 1 Demo: Empty but beautiful dark dashboard shell, API docs visible at `/docs`, database tables exist."

Requirement	Verification	Expected
Dark dashboard shell	Open `http://localhost:3000`	Dark theme, sidebar visible
API docs visible	Open `http://localhost:8000/docs`	Swagger UI with all endpoints
Database tables exist	`docker-compose exec postgres psql -U threatmatrix -c "\dt"`	10 tables
Redis connected	`docker-compose exec redis redis-cli ping`	PONG
Backend running	`curl http://localhost:8000/api/v1/system/health`	`{"status":"operational"}`
Frontend running	`curl http://localhost:3000`	HTTP 200
Navigation works	Click sidebar icons	Routes change
War Room loads	Open `/war-room`	Components render

8.2 Service Status Matrix

Service	Port	Health Check	Expected
PostgreSQL	5432	`pg_isready`	accepting connections
Redis	6379	`redis-cli ping`	PONG
Backend	8000	`/api/v1/system/health`	`{"status":"operational"}`
Frontend	3000	Browser	Page loads with dark theme

Verification:

Check	Command	Expected
All containers up	`docker-compose ps`	2 containers healthy

Check	Command	Expected
Backend healthy	`curl /api/v1/system/health`	Operational
Frontend accessible	`curl localhost:3000`	HTTP 200
API docs accessible	`curl localhost:8000/docs`	HTTP 200
Database tables	`psql -c "\dt"`	10 tables
Redis connected	`redis-cli ping`	PONG

Files Created/Modified Today

```
threatmatrix-ai/
├── backend/
│   ├── app/
│   │   ├── services/
│   │   │   ├── flow_service.py          🔨 TASK 1
│   │   │   └── alert_service.py         🔨 TASK 2
│   │   └── api/
│   │       └── v1/
│   │           ├── flows.py             🔨 TASK 1 (update)
│   │           ├── alerts.py            🔨 TASK 2 (update)
│   │           └── websocket.py         🔨 TASK 3
├── frontend/
│   ├── hooks/
│   │   ├── useWebSocket.ts              🔨 TASK 4 (update)
│   │   ├── useFlows.ts                  🔨 TASK 5
│   │   └── useAlerts.ts                 🔨 TASK 6
│   └── app/
│       └── war-room/
│           └── page.tsx                 🔨 TASK 7 (update)
└── docs/
    └── worklog/
        └── DAY_06_MAR02.md              🔨 This file
```

Verification Checklist

> **Every item below MUST be verified before marking task complete.**

#	Verification	Command	Expected Result
1	Flow service imports	`python -c "from app.services.flow_service import *"`	No errors
2	Alert service imports	`python -c "from app.services.alert_service import *"`	No errors
3	Flow endpoints	Check `/docs`	6 flow endpoints

#	Verification	Command	Expected Result
4	Alert endpoints	Check `/docs`	5 alert endpoints
5	WebSocket endpoint	Check `/docs`	`/ws/` listed
6	Flow CRUD	`curl /api/v1/flows/`	Paginated response
7	Alert CRUD	`curl /api/v1/alerts/`	Alert list
8	Alert stats	`curl /api/v1/alerts/stats`	Statistics object
9	WebSocket connect	`wscat -c ws://localhost:8000/ws/?token=<jwt>`	Connected
10	useWebSocket hook	`grep "export" hooks/useWebSocket.ts`	Function exported
11	useFlows hook	`grep "export" hooks/useFlows.ts`	Function exported
12	useAlerts hook	`grep "export" hooks/useAlerts.ts`	Function exported
13	TypeScript compiles	`cd frontend && npx tsc --noEmit`	No errors
14	War Room live data	Open `/war-room`	Data updates
15	Week 1 demo ready	Full checklist	All items pass

Scope Adherence Verification

Requirement	Source	Verification
Flow Service CRUD	MASTER_DOC_PART2 §5.1	6 endpoints match spec
Alert Service lifecycle	MASTER_DOC_PART2 §5.1	5 endpoints match spec
Alert lifecycle states	MASTER_DOC_PART3 §7.1	open→ack→investigating→resolved
Severity levels	MASTER_DOC_PART3 §7.3	critical/high/medium/low/info
WebSocket channels	MASTER_DOC_PART2 §5.2	flows:live, alerts:live, system:status
Redis pub/sub	MASTER_DOC_PART2 §6.1	Capture → Redis → WebSocket → Browser
RBAC enforcement	MASTER_DOC_PART2 §7.2	Role permissions enforced
useWebSocket hook	MASTER_DOC_PART5 §2.1	Hook interface matches spec
useFlows hook	MASTER_DOC_PART5 §2.1	Hook interface matches spec

Requirement	Source	Verification
useAlerts hook	MASTER_DOC_PART5 §2.1	Hook interface matches spec
API response time	MASTER_DOC_PART1 §10.1	<200ms target
War Room components	MASTER_DOC_PART3 §2.3	All components connected

Blockers

Blocker	Severity	Mitigation	Status
Next.js 16 build error	Medium	Use `npm run dev` for development	Known bug
No LLM API keys yet	Low	Mock responses for now	Expected
No threat intel keys yet	Low	Mock data for now	Expected

Tomorrow's Preview (Day 7 — Week 2 Start)

Per MASTER_DOC_PART5 §3 Week 2 plan:

- Scapy capture engine: packet sniffing, flow aggregation
- Feature extraction pipeline (40+ features)
- Redis pub/sub integration (capture → Redis → API)
- War Room: ThreatMap (Deck.gl + Maplibre)
- MetricCard, StatusBadge, DataTable components

Reference Documents

Document	Section	Purpose
MASTER_DOC_PART2	§5.1	API endpoints specification
MASTER_DOC_PART2	§5.2	WebSocket events
MASTER_DOC_PART2	§6.1	Redis pub/sub architecture
MASTER_DOC_PART2	§7.2	RBAC permissions matrix
MASTER_DOC_PART3	§2.3	War Room component specs
MASTER_DOC_PART3	§7.1	Alert lifecycle
MASTER_DOC_PART3	§7.3	Severity definitions
MASTER_DOC_PART5	§2.1	Project structure (hooks)
MASTER_DOC_PART5	§3	Week-by-week plan

Task workflow for Day 6 — ThreatMatrix AI Sprint 1